Diagnosing a failed scaling or training decision

Diagnose the first broken contract

When a larger run underperforms, the cause can be scientific, numerical, data-related or infrastructural. Preserve the checkpoint and complete lineage before changing multiple knobs. A useful diagnosis makes a discriminating prediction: what observation would support or reject this cause?

Symptom	Plausible cause	First discriminating check	What to keep fixed
Forecast misses at larger size	HP transfer failure, model-form error, count mismatch	Reconcile actual tensor/FLOP/token definitions; inspect endpoint schedule and held-out residuals	Evaluator and data generation
New mixture lowers own training loss only	Easier target mixture	Evaluate both models on the same held-out distribution	Scored text, masks, context and task weights
Quality filtering causes repeated-data degradation	Retained supply shrank; weights unchanged	Recompute cumulative exposure by cell using retained counts	Tokenizer and phase budgets
“Same” bucket weights behave differently	Taxonomy/scorer generation changed	Compare centroid bytes, calibration and cell-to-topic mapping	Source/store identities
Wider model becomes unstable	Parameter-group convention or initialization changed	Emit actual group LRs, norms, clipping and activation/update diagnostics	Depth, horizon, batch and schedule
Larger batch changes behavior unexpectedly	Update-count warmup, decay or momentum token-timescale changed	Convert every update-based setting into tokens; inspect actual LR/decay path	Token horizon and data order
Token likelihood improves but task accuracy does not	Near-chance accuracy, format/export issue, different target	Check answer likelihood, generated samples and native/export parity	Task revision and prompt/extractor
Longer-context score improves suddenly	Additional conditioning or hidden truncation change	Evaluate old fixed context and per-length generation coverage	Checkpoint and tokenizer
SFT initializes but optimizer update fails	Model/tokenizer padding or parameter-tree mismatch	Compare parameter and optimizer-state shapes after initialization	Exact model-source branch
A formerly fitting MoE OOMs	Unsharded optimizer workspace, routing buffer or logits tile	Profile peak allocation and sharding of that tensor	Sequence length, batch and precision
An issue is closed but no score is visible	Experiment tracking status differs from scientific completion	Find runtime SHA, final checkpoint and actual evaluation artifact	Avoid inferring a winner
A cyber/RL probe improves but broad tasks regress	Specialized data/reward budget or generation-length tradeoff	Base/control/treatment on independent families and guardrails	Evaluation environment and attempt budget

These are candidate diagnoses, not universal rankings of likelihood. The hypothesis sources and Marin case study explain which failure modes are directly documented and which checks are our synthesis.

A controlled recovery sequence

1. Establish whether the run actually trained and evaluated to the intended endpoint. A scheduled job, a launched worker gang and a completed scored run are different events.
2. Reproduce the metric from saved per-item or numerator/denominator records. Verify evaluator version, coverage, routing mode and context.
3. Reconcile consumed tokens and data order with parent checkpoint and sampler offsets. A context switch can change sequence/block interpretation and cause repeated traversal.
4. Compare resolved optimizer groups and state, rather than only config base LR. Check dtype, initialization, schedule boundary and recurrence visit count.

5. Test one cause on a small replay or matched treatment. Keep an unchanged control and preserve the failed configuration.
6. If the larger holdout influenced a recipe revision, mark it as development evidence and reserve a new independent check.

The 2×long-context experiment discussion documents a worker-registration problem that initially prevented training. Charging that pending time to a data-mixture throughput comparison would be misleading. The SFT geometry PR documents a different issue: optimizer workspace and sharding constrain memory even when total parameter counts look feasible.

Evidence needed to close a diagnosis

Record the incident ID, source/runtime SHAs, checkpoint and consumed-token state, symptom/metric, proposed cause, controlled intervention, measured before/after result and remaining alternatives. A code fix plus a green smoke can close an infrastructure bug while leaving the scientific recipe unvalidated. Keep those conclusions separate.

For a new cyber task or recurrent architecture, expose unresolved assumptions in the question contract instead of declaring the problem solved by analogy with a different model. Current gaps are part of the knowledge base.